

Feature: Leads list and contact page

From build-plan: feature 11

Size: heavy - two separate risks with real logic: showing one business's customers to its login and never another's (security), and a write path that creates contacts and leads by hand (data).

Branch: `feature/11-leads-list-and-contact-page`

Status: verified 2026-10-10: both steps built and reviewed; backend 901, shared 172 and booking component 79 tests pass, the frontend builds and lints.

Goal

The owner signs in and sees every lead the business has, newest first, with its stage; opens one and reads the person behind it, that request, the person's other requests, their whole timeline and their open next steps; and types in a lead that came by phone, so the painter's notebook goes into the CRM instead.

Design reference

- `prototypes/leads.html` (the list, inside the app shell of `prototypes/shell.css`)
- `prototypes/contact.html` (the contact page and its timeline)
- Tokens already ported to `frontend/app/globals.css` from `prototypes/theme.css`.

Only what this feature has data for is built from them. The stats tiles, Value column, Export, search and filters, quotes, the email composer, "Log a call" and "Book a time" belong to later features (see Out of scope).

In scope

- **The app shell:** a sidebar with the business mark and name, nav to **Leads** and **Setup** (today's home page cards), and sign out. Shared by `/`, `/leads` and `/leads/[leadId]`, with one `/me` gate (signed out, pick a business, plan refused, API down) instead of each page repeating it.
- **The leads list** at `/leads`: every lead of the signed-in business, newest first, 50 at a time with "Show more". Each row: the person's initials, name, phone or email; what they asked for (the booked service, else the first line of their own words, else a dash); when (the booking's time in the business's time zone, marked cancelled when it is, else when the lead came in); the stage; where it came from (Website, Booking page, Added by hand). A row opens the lead's page.
- **The lead page** at `/leads/[leadId]`: the person (name, phone as a `tel:` link, email as a `mailto:` link); this request (stage, source, when it came in, the phone given with it, their words, the answers to the business's own questions, its bookings with service, time, person, address and status); the

person's other requests, each linking to its page; **open next steps** (due, not done, soonest first); and **the timeline**, newest first.

- **Adding a lead by hand** (step 11.2): an "Add a lead" button on the list opens a form; saving makes the lead in the first stage, joins an existing contact when the email is already known, writes "Added by hand" on the timeline, and opens the new lead's page.
- Loading, empty, not-found, refused and API-down states for every screen.

Out of scope

- Changing a lead's stage, renaming or reordering stages (feature 14, the board).
- Editing or deleting a contact or a lead; merging duplicates.
- Notes, logging a call, sending email or texts from the page (feature 15).
- Quotes, money columns and the stats tiles (features 16, 20).
- Search, filters and export (not planned yet; a note for later).
- The owner booking a time for a lead (feature 12b puts things on calendars).
- TanStack Query: the build plan brings it at feature 14 with the board; these screens use the fetch-and-state pattern every dashboard screen uses today.

Build loop

Heavy, so every step: its plan gets Frank's yes just before it is built; after the yes nothing stops until the review (build, tests and checks, tick the box, write the log, commit and push to the feature branch, `/audit` scoped to the step, independent review). The stop is after the review, where findings are talked through; P0/P1 are fixed or accepted by Frank with a reason, P2/P3 are recorded and carried. `/complete` does the final pass and the merge on his yes.

Build steps

- ☒ **11.1 The leads list and the lead page.** Two read routes scoped to the session's business, the app shell, `/leads` and `/leads/[leadId]`.
 - 11.1.1 `GET /leads?after=<leadId>`: the business's leads, newest first (createdAt, then id), 50 per page, `nextAfter` when there are more. The business comes only from the session; `after` must be one of its own leads or the answer is 400.
 - 11.1.2 `GET /leads/:leadId`: one lead with its contact, its bookings, the contact's other leads, open next steps and the timeline (the newest 200, with `timelineCut: true` when there are more). Another business's lead and an unknown id answer the same 404.
 - 11.1.3 The timeline view: the backend turns each activity row into a typed entry with only the facts the screen shows (see Data / contracts); the raw payload never crosses the API, and a type with no

view yet still shows as its label and time.

- 11.1.4 The shell and the `/me` gate, shared by `/`, `/leads` and `/leads/[leadId]`; the home page's cards move under Setup unchanged.
- 11.1.5 The two screens, with loading, empty, not-found and API-down states.
- **Done when:** one backend test proves a business never sees another's leads (the clinic owner gets 404 for a painting lead, and the list never holds one); `npm run build --workspace=frontend` passes; and in the browser at `http://localhost:3400/leads` a booking made through the booking window shows as the top row, and opening it shows the booking, the email and text entries and the "Booked" line on its timeline.
- ☒ **11.2 Adding a lead by hand.** The form, `POST /leads`, the new timeline type.
 - 11.2.1 `lead_added` joins `ACTIVITY_TYPES`, and `lead` gets a nullable `requestKey`, unique per business; one migration for both.
 - 11.2.2 `POST /leads`, in one transaction: find or make the contact by the existing rule, the lead in the first stage with source `manual`, and the `lead_added` entry with the login as its actor. The same `requestKey` sent twice gives back the first lead, never a second one.
 - 11.2.4 Next steps (Frank, Oct 10, option A): on the lead page, add one (a few words and when it is due) as a `task` activity on the contact, and tick it done. `POST /leads/:leadId/next-steps`, `POST /leads/:leadId/next-steps/:nextStepId/done`, both scoped through the lead to the session's business; another business's answers 404.
 - 11.2.3 The form on `/leads`: name, phone, email, what they want; saving opens the new lead's page, which says so when the person was already a contact.
 - **Done when:** four backend tests prove: a known email joins that person, no email makes a new person, the same form sent twice makes one lead, and a lead with neither phone nor email is refused; and in the browser a lead typed in at `http://localhost:3400/leads` appears at the top and its page shows "Added by hand" with the login's name.

Files / areas

Backend (11.1):

- `backend/app.ts` - mount `/leads` with dashboard CORS, CSRF and no-store, as `/email-sending` is; `.route("/leads", leadsRoutes)` in the one chain.
- `backend/routes/leads-routes.ts` - `GET /`, `GET /:leadId` (11.2 adds `POST /`), each behind `requireOrganizationMiddleware`, `requireKnownSubscriptionMiddleware`, `requireModuleMiddleware("crm")`.
- `backend/lib/crm/find-leads-page.ts`, `backend/lib/crm/find-lead-page.ts`, `backend/lib/crm/timeline-entry-of.ts` - one export each.
- Their `*.test.ts` beside them, against the seeded `scheduleads_dev`.

Backend (11.2):

- `packages/shared/crm/activity-types.ts` - add `lead_added` .
- `packages/shared/db/crm-tables/lead-table.ts` - `requestKey` , unique index on (organizationId, requestKey) where not null.
- `packages/shared/migrations/` - one generated migration.
- `packages/shared/zod-validation/crm-validation-schemas/add-lead-validation-schema.ts`
- `backend/lib/crm/add-lead-by-hand.ts` and its test.

Frontend:

- `frontend/components/dashboard/dashboard-gate.tsx` (the `/me` switch lifted out of `app/page.tsx`) and `dashboard-shell.tsx` (the sidebar).
- `frontend/app/page.tsx` - uses the gate and shell; its cards unchanged.
- `frontend/app/leads/page.tsx` , `frontend/app/leads/[leadId]/page.tsx` .
- `frontend/components/leads/leads-list-screen.tsx` , `lead-page-screen.tsx` , `timeline-entry-line.tsx` ; 11.2 adds `add-lead-form.tsx` .
- `frontend/lib/api-client/leads/fetch-leads.ts` , `fetch-lead.ts` ; 11.2 adds `add-lead.ts` .
Every call through `dashboardApiClient` .

Data / contracts

Who may do what. Any member of the business (owner, admin, member) whose plan includes the `crm` module may read its leads and add one. Adding a lead destroys nothing and a receptionist takes calls, so no role check. The business is always the session's (`c.get("organization").organizationId`), never a request value, and every query filters by it first.

GET `/leads?after=<leadId>` → 200

```
{
  leads: {
    id: string;
    createdAt: string;           // ISO
    source: "widget" | "hosted" | "manual";
    stage: { id: string; name: string };
    contact: { id: string; name: string; phone: string | null; email: string | null };
    what: string | null;         // booked service name, else first line of deta
    booking: { startsAt: string; status: "confirmed" | "cancelled" } | null; // the le
  }[];
  nextAfter: string | null;     // the last lead's id when more exist
  timeZone: string | null;      // the business's, from its business-wide avail
}
```

Phone shown is the lead's own phone, else the contact's. Errors: `400 bad_request` for an `after` that is not one of the business's leads; `401`, `403 forbidden`, `403 plan_required` from the existing middleware.

`GET /leads/:leadId` → `200`

```
{
  lead: { id, createdAt, source, stage: { id, name }, phone: string | null,
    details: string | null, answers: { question: string; answer: string }[] };
  contact: { id, name, email: string | null, phone: string | null };
  bookings: { id, serviceName: string, startsAt: string, endsAt: string,
    status: "confirmed" | "cancelled", personName: string,
    placeName: string | null, location: string | null }[]; // newest first
  otherLeads: { id, createdAt, stageName: string, what: string | null }[]; // newest f
  nextSteps: { id, type: string, dueAt: string, what: string | null }[]; // dueAt set
  timeline: TimelineEntryType[]; // occurredAt set, newest first, at most 200
  timelineCut: boolean;
  timeZone: string | null;
}
```

`404 not_found` ("No lead here.") for an unknown id and for another business's lead alike, so the answer never tells whether it exists elsewhere.

TimelineEntryType - `{ id, type, occurredAt, actorName: string | null }` plus, per type, only these facts (looked up within the same business):

| type | facts | the line reads | ---|---|---| | `booking_created` | serviceName, startsAt | Booked {service} for {time} | | `booking_moved` | serviceName, fromStartsAt, toStartsAt | Moved {service} from {time} to {time} | | `booking_cancelled` | serviceName | Cancelled {service} | | `email_sent` | kind | Sent the confirmation / the business was told / ... | | `sms_sent` | kind, minutesBefore | Texted the confirmation / a reminder {n} before | | `later_texts_yes` | none | Said yes to texts | | `lead_added` (11.2) | none | Added by hand by {actorName} | | any other | none | its label |

Twilio and Resend ids, phone numbers inside payloads and the customer's words in a `later_texts_yes` payload are not sent. Every string is rendered as text, never HTML. Times are shown in the business's time zone; with none set, in the browser's, labelled as such.

POST /leads (11.2) - body `{ requestKey: uuid, name: string, phone?: string, email?: string, details?: string }`. Name 1-120, phone up to 40, email by the existing email rule, details up to 2000; at least one of phone or email ("Enter a phone number or an email, so you can reach them."). Contact by `findOrCreateContact` unchanged: a known email joins that contact and keeps its name and phone (a phone is filled in only when it had none); no email is always a new contact; a phone never matches. Lead: first stage, source `manual`, `details`, `phone`, `answers` null, `requestKey`. Activity `lead_added`, payload `{ leadId }`, actor the login. One transaction. → `201 { leadId, contactId,`

`joinedExistingContact }` ; the same `requestKey` again → `200` with the first lead's answer. `400 bad_request` with the first field's message; `401 / 403` as above.

Testing

A test only where a mistake would hurt (workspace rule, Oct 10): five in all, with Vitest against the seeded local `scheduleads_dev` , each adding and removing its own rows.

- 11.1: a business never sees another's leads.
- 11.2: a known email joins that person; no email makes a new person; the same form sent twice makes one lead; neither phone nor email is refused.
- The existing "one of each type" activity test covers `lead_added` once it is in the list.
- `npm run build --workspace=frontend` for the typed client and the screens; the frontend has no test runner and gets none here.
- By hand in the browser at `http://localhost:3400`, per each step's Done when.

Notes for the AI

- Next 16: read `node_modules/next/dist/docs/` for dynamic route params in a client page before writing `app/leads/[leadId]/page.tsx` .
- The business time zone lives on the business-wide `availability_rule` row (`personId` null); `find-booking-email-context.ts` already reads it. Reuse that query's shape; never assume one.
- A lead normally has at most one booking (moves change the same row), but the schema allows more: always take the newest.
- `requestKey` follows the booking form's pattern (`booking.requestKey` , `one-copy-of-a-form-at-a-time.ts`): the form makes one uuid when it opens.
- Never log a contact's name, phone or email; errors go through `safeErrorReason` .
- `/leads` and `/leads/*` both need the dashboard middleware; prove with a test that a cross-site POST to `/leads` is refused by the CSRF check.

Open questions

None. Writing next steps was answered on Oct 10: yes, in 11.2 (option A).

Implementation walkthrough

Feature 11 was specced heavy with two steps and kept them. On the same day, while planning, Frank set the workspace rule on cutting steps: parts are for explaining, a step is only something he can see working on its own, and tests go only where a mistake would really hurt. The spec's 19 planned checks became 5;

step 11.1 got one saved test, step 11.2 six, and the review fixes one more. On Frank's answer to the open question (option A), 11.2 also gained next steps.

backend: reading leads, one business at a time (11.1)

`routes/leads-routes.ts` holds every `/leads` route behind the login, the plan check and the `crm` module; `app.ts` mounts `/leads` and `/leads/*` with the dashboard CORS, cross-site and no-store middleware. Any member of the business may read and add: nothing here destroys anything.

`lib/crm/find-leads-page.ts` pages the list 50 at a time, newest first, by `(createdAt, id)`; the cursor is compared inside Postgres with a subquery, because a JavaScript date drops the microseconds `createdAt` keeps. An `after` that is not one of the business's leads is a 400. `lib/crm/find-lead-page.ts` reads one lead with its contact, bookings (service, person, room, address, status), the contact's other leads, open next steps and the newest 200 timeline rows; another business's lead is the same null as one that never existed, so the route answers one 404. `lib/crm/timeline-entry-of.ts` turns each activity row into a typed entry with only the facts its line shows, so Twilio and Resend ids, phone numbers and a customer's words never leave the API. Small helpers shared by both: `find-business-time-zone.ts` (the business-wide availability rule's zone), `find-newest-bookings.ts` and `lead-what.ts`.

frontend: the first CRM screens (11.1)

The signed-in pages moved into `app/(dashboard)/` under one layout: `DashboardGate` asks `/me` once and shares it through `useMe()`, and `DashboardShell` draws the sidebar (the business, Leads, Setup, sign out), so moving between screens keeps both mounted (F-312). Setup is the old home page's cards, unchanged. `components/leads/` holds the list (`leads-list-screen.tsx`), the lead page (`lead-page-screen.tsx`) and the timeline line, whose wording keys on the kinds the backend actually saves (F-311). Every call goes through the typed dashboard client in `lib/api-client/leads/`; an ended sign-in and a refusal each get their own notice (F-313). Times show in the business's zone, or the browser's with a note when the business has none.

adding a lead and next steps by hand (11.2)

Migration 0026 adds `lead_added` to the timeline types and `lead.requestKey`, unique per business when set. `lib/crm/add-lead-by-hand.ts` runs in one transaction: the contact by the rule bookings use (a known email is that person, name and phone kept, a phone filled in only when they had none; no email is always new; a phone never matches), the lead in the first stage with source `manual`, and "Added by hand" with the login as actor. A second save of the same form loses on the request key's index, rolls back (taking any contact it made with it) and answers with the first lead; a repeat works out "already a contact" from the two creation times (F-316). The form (`add-lead-form.tsx`) makes its key when it opens and puts the phone-or-email message under Phone. Next steps are `task` activities on the contact: `add-next-step.ts` reads the typed clock time in the business's zone, else the browser's, with the tested `localTimeToMoment`, refusing a skipped spring hour and an impossible date (F-314); `finish-next-step.ts` ticks one done only through a lead of the same business. Both wrap database errors (F-315).

Proof

Saved tests in `backend/routes/leads-routes.test.ts`: a business never sees another's leads; a known email joins that person; no email makes a new person; the same form sent twice makes one lead (both sends at once); neither phone nor email is refused; next steps stay inside the business; a cross-site post is refused. Three were made to fail on purpose (the business filter on the lead page, the next-step scope, the request-key guard). By hand at `http://localhost:3400/leads`: a booking made in the booking window was the top row with its page and timeline; a lead typed in for a known email joined her; a next step was added, shown at the time typed, and ticked done. Not seen live: the email and text lines, because the dev business sends no email and the test booking gave no phone.

Findings

11/F-311 [P2] closed - every text on the timeline read "Sent a text"

File: `frontend/components/leads/timeline-entry-line.tsx:18-23` **Found:** 2026-10-10 by the independent review of step 11.1 (b680844) **Why it matters:** The screen keyed texts on `confirmation`, `reminder`, `added`, `moved`, `removed`; the backend saves `booking_confirmation`, `booking_reminder` and `worker_added|moved|removed` (`send-booking-text.ts:55`, `send-worker-text.ts:89`), so no text ever got its own line. **Suggested fix:** Key the lines on the saved kinds. **Resolution:** fixed in e11c7d0. Closed 2026-10-10 by the final independent review of feature 11 (main...648ba0c): the fix is present and correct.

11/F-312 [P3] closed - the whole frame reloaded on every move between screens

File: `frontend/app/page.tsx`, `frontend/app/leads/page.tsx`, `frontend/app/leads/[leadId]/page.tsx` **Found:** 2026-10-10 by the independent review of step 11.1 (b680844) **Why it matters:** Each page mounted its own sign-in check and sidebar, so every click asked /me again and flashed "Loading your business...". **Suggested fix:** One shared layout for the signed-in pages. **Resolution:** fixed in 8202904 and e11c7d0: the pages moved into `app/(dashboard)/` under one layout; pages read the business with `useMe()`. URLs unchanged. Closed 2026-10-10 by the final independent review of feature 11 (main...648ba0c): the fix is present and correct.

11/F-313 [P3] closed - an ended sign-in showed as "unexpected status (401)"

File: `frontend/lib/api-client/leads/fetch-leads.ts`, `fetch-lead.ts` **Found:** 2026-10-10 by the independent review of step 11.1 (b680844) **Why it matters:** The spec asks for a refused state on every screen; a 401 or 403 fell into "API down" with a Try again that could not work. **Suggested fix:** Map 401 to a sign-in-again notice and 403 to its message. **Resolution:** fixed in e11c7d0. Closed 2026-10-10 by the final independent review of feature 11 (main...648ba0c): the fix is present and correct.

11/F-314 [P2] closed - a next step's due time was read in the browser's zone, shown in the business's

File: frontend/components/leads/next-steps-section.tsx:39 **Found:** 2026-10-10 by the independent review of step 11.2 (b7bc788) **Why it matters:** An owner whose browser is in another zone typed 9:00 and saw "due 10:00". **Suggested fix:** Read the typed time in the business's zone. **Resolution:** fixed: the form sends the clock time it was given and the browser's zone; the API reads it in the business's zone (the browser's when the business has none, as the page shows it) with the tested `localTimeToMoment`; a skipped spring hour and an impossible date are refused. Closed 2026-10-10 by the final independent review of feature 11 (main...648ba0c): the fix is present and correct.

11/F-315 [P3] closed - next-step database errors were not wrapped

File: backend/lib/crm/add-next-step.ts, finish-next-step.ts **Found:** 2026-10-10 by the independent review of step 11.2 (b7bc788) **Why it matters:** Hono logs an unwrapped database error whole, query and owner's words included. **Resolution:** fixed: both go through `safeErrorReason`. Closed 2026-10-10 by the final independent review of feature 11 (main...648ba0c): the fix is present and correct.

11/F-316 [P3] closed - a repeated save always said the person was new

File: backend/lib/crm/add-lead-by-hand.ts **Found:** 2026-10-10 by the independent review of step 11.2 (b7bc788) **Why it matters:** A retry after a lost answer dropped the "Already a contact" notice. **Resolution:** fixed: a repeat compares the contact's and the lead's creation times (a contact made with the lead shares its transaction's timestamp). Closed 2026-10-10 by the final independent review of feature 11 (main...648ba0c): the fix is present and correct.

11/F-317 [P3] closed - the spec's cross-site test was missing

File: backend/routes/leads-routes.test.ts **Found:** 2026-10-10 by the independent review of step 11.2 (b7bc788) **Why it matters:** Nothing was exposed (the check is mounted); only the proof the spec asked for was absent. **Resolution:** fixed: "a cross-site post to /leads is refused". Closed 2026-10-10 by the final independent review of feature 11 (main...648ba0c): the fix is present and correct.

Independent review

Status: passed **Target commit:** 648ba0cbe47a43d78136f04af886454793045638 **Base commit:** 31ee18682d34e3b6f2eff957f8b010d85a6a86f4 **Base ref:** main **Spec hash:** 75487ed41d89ba63da51b30a52aec33a904aaa38fb602a6537cb55441f12508e **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** runtime default **Requested execution:** automatic **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent, read-only, one per review **Actual execution:** automatic **Reviewed at:** 2026-10-10 **Scope:** step 11.1 (b680844), step 11.2 (b7bc788), then the whole feature (main...648ba0c) **Lenses:** tenant scope, data leaks, idempotency and races, validation, correctness **Verdict:** passed **Check result:** not-required

Rounds

1. Step 11.1 at b680844: no P0 or P1. Tenant scope clean on every query, the cursor subquery included; no payload, provider id or phone leaves the API. Found F-311 (P2), F-312, F-313 (P3), fixed in 8202904 and e11c7d0. 2. Step 11.2 at b7bc788: no P0 or P1. Tenant scope held on every read and write; the double save is settled by the database and the losing save's contact rolls back. Found F-314 (P2), F-315, F-316, F-317 (P3), fixed in 648ba0c. 3. Final integration review of main...648ba0c: F-311 to F-317 each verified closed; every `/leads` route behind the dashboard middleware and the `crm` module; no page outside the shared layout; nothing broken by the fix commits. Raised F-318 and F-319 (P3, missing tests), carried on purpose.

The audit gate was met by these reviews rather than a separate `/audit` pass, and the receipt is recorded here directly instead of through `blueprint/context/review.md`, as for feature 10.

Commands

At 648ba0c: `npm run build --workspace=backend`, `npm run build --workspace=frontend`, `npm run lint --workspace=frontend`, `npm run test --workspace=@scheduleads-app/shared` (172 passed), `npm run test --workspace=backend` (901 passed), `npm run test --workspace=@frankdmosquera/booking-component` (79 passed).

Remaining risk

The email and text lines on the timeline were not seen live (no email or phone in the dev test). F-318 and F-319 leave the due-time zone choice and the repeat-save flag without saved tests.